

Laboratorio di analisi campioni

Esercizio di riepilogo — Advanced Computer Programming · Python: OOP, eccezioni, decoratori, generatori e concorrenza

L'esercizio simula un laboratorio che riceve campioni, li analizza con un numero limitato di strumenti e produce un report aggregato. È strutturato in tre parti: la prima richiama i costrutti delle lezioni precedenti, la seconda introduce il multithreading e la sincronizzazione, la terza serve a verificare la comprensione dei meccanismi usati. Le parti sono pensate per essere svolte in ordine, perché la Parte 2 riusa direttamente le classi definite nella Parte 1.

Parte 1 — Modello a oggetti

Definisci in un modulo `laboratorio.py` le entità descritte di seguito.

Classe `Campione`

Classe base con attributi `codice: str`, `volume_ml: float`, `priorita: int`. Deve rispettare i seguenti requisiti:

- `volume_ml` esposto come `@property`, con setter che solleva `VolumeNonValidoError` — eccezione personalizzata, sottoclasse di `ValueError` — quando il valore è minore o uguale a zero.
- `__repr__` leggibile e `__lt__` basato sulla priorità, in modo che una lista di campioni sia ordinabile con `sorted()` senza specificare `key`.
- `__eq__` e `__hash__` coerenti fra loro e definiti sul solo `codice`, così che i campioni possano essere inseriti in un `set` senza duplicati logici.
- Un factory method `@classmethod da_riga(cls, riga: str)` che costruisce l'oggetto a partire da una riga in formato CSV come `"C001;12.5;3"`.

Sottoclassi `CampioneEmatico` e `CampioneChimico`

Ridefiniscono il metodo `analizza(self) -> dict`, ciascuna calcolando metriche differenti e simulando il tempo di elaborazione con `time.sleep()` proporzionale al volume. Nella classe base `analizza` è dichiarato come `@abstractmethod` ereditando da `abc.ABC`: istanziare direttamente `Campione` deve fallire.

Decoratore `@cronometra`

Misura il tempo di esecuzione del metodo decorato e inserisce il valore nel dizionario restituito sotto la chiave `"durata_s"`. Usa `functools.wraps` per preservare nome e docstring della funzione originale, e verificalo stampando `CampioneEmatico.analizza.__name__`.

Generatore `leggi_campioni(percorso)`

Apri il file con un blocco `with`, scarta le righe malformate gestendo l'eccezione senza interrompere l'iterazione, e produce con `yield` un `Campione` alla volta. Il punto è che la lettura resti *lazy*: il file non deve mai essere caricato interamente in memoria.

Parte 2 — Concorrenza

2.1 Produttore/consumatore

Un thread produttore consuma il generatore e inserisce i campioni in una `queue.Queue` con `maxsize=10`; N thread consumatori prelevano dalla coda, invocano `analizza()` e accumulano i risultati. Termina i worker con il pattern *sentinella* (un valore `None` in coda per ciascun worker) e chiudi la fase con `queue.join()` abbinato alle chiamate a `task_done()`.

2.2 Risorsa limitata

Il laboratorio dispone di soli tre spettrometri: proteggi la sezione di analisi con un `threading.Semaphore(3)` usato come context manager. Verifica sperimentalmente che non più di tre thread si trovino contemporaneamente nella sezione critica, incrementando e decrementando un contatore ausiliario e inserendo un `assert`.

2.3 Stato condiviso

Realizza una classe `Statistiche` con i campi `campioni_elaborati`, `volume_totale` e un dizionario dei conteggi per tipo di campione. Implementala **prima senza lock**, eseguila su circa duemila campioni e osserva la race condition sui contatori; correggila poi con un `threading.Lock` interno alla classe, in modo che la sincronizzazione sia un dettaglio di implementazione e non una responsabilità del chiamante.

2.4 Coordinamento e terminazione

Aggiungi un thread supervisore che ogni mezzo secondo stampa lo stato di avanzamento e si arresta quando un `threading.Event` viene impostato dal main al termine del lavoro. Il supervisore deve essere creato con `daemon=False` e terminare in modo pulito, senza ricorrere a interruzioni forzate.

2.5 Sincronizzazione a basso livello

Sostituisci la `Queue` con una classe `BufferLimitato` scritta a mano usando `threading.Condition`, con i metodi `inserisci()` e `preleva()` basati su `wait()` e `notify_all()`. L'obiettivo è capire quale meccanismo la `Queue` della libreria standard incapsula.

2.6 Variante con executor

Riscrivi la sola fase di analisi utilizzando `concurrent.futures.ThreadPoolExecutor` con `submit()` e `as_completed()`, gestendo le eccezioni sollevate all'interno dei worker attraverso `future.result()`. Confronta la leggibilità del risultato con la versione di 2.1.

Parte 3 — Domande di consolidamento

Rispondi in poche righe, senza scrivere codice.

#	Domanda
1	Perché al punto 2.3 la race condition si manifesta anche se <code>campioni_elaborati += 1</code> occupa una sola riga di codice?
2	Questo carico di lavoro trae beneficio dal multithreading? Cosa cambierebbe se l'analisi fosse puramente CPU-bound, e quale strumento useresti in quel caso?
3	Qual è la differenza fra <code>Lock</code> e <code>RLock</code> , e in quale punto del tuo codice un <code>Lock</code> semplice porterebbe a un deadlock?
4	Cosa accade se ometti la chiamata a <code>task_done()</code> ? E se un worker solleva un'eccezione prima di eseguirla?
5	Perché il pattern sentinella richiede esattamente un valore <code>None</code> per ciascun worker e non uno soltanto?

Suggerimento di svolgimento. Fai girare e collauda la Parte 1 in isolamento prima di introdurre i thread: se il modello a oggetti non è solido, i bug di concorrenza diventano indistinguibili dai bug di logica e il debugging si complica notevolmente.